

Skipping ads...

API > indicators

nb

numba

 λ nb module 

Numba-compiled functions for custom indicators.

Provides an arsenal of Numba-compiled functions that are used by indicator classes. These only accept NumPy arrays and other Numba-compatible types.

adx_1d_nb function 

```
adx_1d_nb(  
    high,  
    low,  
    close,  
    window=14,  
    wtype=3,  
    minp=None,  
    adjust=False  
)
```

Average Directional Movement Index (ADX).

Returns +DI, -DI, DX, and ADX.

For `wtype`, see [WType](#).

adx_nb function 

```
adx_nb(  
    high,  
    low,  
    close,  
    window=14,  
    wtype=3,  
    minp=None,  
    adjust=False  
)
```

```
    high,  
    low,  
    close,  
    window=14,  
    wtype=3,  
    minp=None,  
    adjust=False  
)
```

2-dim version of [adx_1d_nb](#).

atr_1d_nb function

```
atr_1d_nb(  
    high,  
    low,  
    close,  
    window=14,  
    wtype=3,  
    minp=None,  
    adjust=False  
)
```

Average True Range (ATR).

Returns TR and ATR.

For `wtype`, see [WType](#).

atr_nb function

```
atr_nb(  
    high,  
    low,  
    close,  
    window=14,  
    wtype=3,  
    minp=None,
```

```
        adjust=False  
    )
```

2-dim version of [atr_1d_nb](#).

avg_gain_1d_nb function

```
avg_gain_1d_nb(  
    close,  
    window=14,  
    wtype=3,  
    minp=None,  
    adjust=False  
)
```

Average gain.

avg_gain_nb function

```
avg_gain_nb(  
    close,  
    window=14,  
    wtype=0,  
    minp=None,  
    adjust=False  
)
```

2-dim version of [avg_gain_1d_nb](#).

avg_loss_1d_nb function

```
avg_loss_1d_nb(  
    close,  
    window=14,
```

```
wtype=3,  
minp=None,  
adjust=False  
)
```

Average loss.

avg_loss_nb function

```
avg_loss_nb(  
    close,  
    window=14,  
    wtype=0,  
    minp=None,  
    adjust=False  
)
```

2-dim version of [avg_loss_1d_nb](#).

bbands_1d_nb function

```
bbands_1d_nb(  
    close,  
    window=14,  
    wtype=0,  
    alpha=2.0,  
    minp=None,  
    adjust=False,  
    ddof=0  
)
```

Bollinger Bands.

Returns the upper band, the middle band, and the lower band.

For `wtype`, see [WType](#).

bbands_bandwidth_1d_nb function

```
bbands_bandwidth_1d_nb(  
    upper,  
    middle,  
    lower  
)
```

Bollinger Bands Bandwidth.

bbands_bandwidth_nb function

```
bbands_bandwidth_nb(  
    upper,  
    middle,  
    lower  
)
```

2-dim version of [bbands_bandwidth_1d_nb](#).

bbands_nb function

```
bbands_nb(  
    close,  
    window=14,  
    wtype=0,  
    alpha=2.0,  
    minp=None,  
    adjust=False,  
    ddof=0  
)
```

2-dim version of [bbands_1d_nb](#).

bbands_percent_b_1d_nb function

```
bbands_percent_b_1d_nb(  
    close,  
    upper,  
    lower  
)
```

Bollinger Bands %B.

bbands_percent_b_nb function

```
bbands_percent_b_nb(  
    close,  
    upper,  
    lower  
)
```

2-dim version of [bbands_percent_b_1d_nb](#).

final_basic_bands_nb function

```
final_basic_bands_nb(  
    close,  
    upper,  
    lower,  
    prev_upper,  
    prev_lower,  
    prev_direction  
)
```

Final bands at one iteration.

get_dma_hurst_nb function

```
get_dma_hurst_nb(  
    close,  
    min_chunk=8,  
    max_chunk=100,  
    num_chunks=5  
)
```

Estimate the Hurst exponent using DMA method.

Windows are linearly distributed.

get_dsod_hurst_nb function

```
get_dsod_hurst_nb(  
    close  
)
```

Estimate the Hurst exponent using discrete second order derivative.

get_hurst_nb function

```
get_hurst_nb(  
    close,  
    method=0,  
    max_lag=20,  
    min_log=1,  
    max_log=2,  
    log_step=0.25,  
    min_chunk=8,  
    max_chunk=100,  
    num_chunks=5,  
    stabilize=False  
)
```

Estimate the Hurst exponent using various methods.

Uses the following methods:

- `HurstMethod.Standard` : `get_standard_hurst_nb`
- `HurstMethod.LogRS` : `get_log_rs_hurst_nb`
- `HurstMethod.RS` : `get_rs_hurst_nb`
- `HurstMethod.DMA` : `get_dma_hurst_nb`
- `HurstMethod.DSOD` : `get_dsod_hurst_nb`

`get_log_rs_hurst_nb` function 

```
get_log_rs_hurst_nb(  
    close,  
    min_log=1,  
    max_log=2,  
    log_step=0.25  
)
```

Estimate the Hurst exponent using R/S method.

Windows are log-distributed.

`get_rs_hurst_nb` function 

```
get_rs_hurst_nb(  
    close,  
    min_chunk=8,  
    max_chunk=100,  
    num_chunks=5  
)
```

Estimate the Hurst exponent using R/S method.

Windows are linearly distributed.

get_rs_nb function 

```
get_rs_nb(  
    close  
)
```



Get rescaled range (R/S) for Hurst exponent estimation.

get_standard_hurst_nb function 

```
get_standard_hurst_nb(  
    close,  
    max_lag=20,  
    stabilize=False  
)
```



Estimate the Hurst exponent using standard method.

iter_basic_bands_nb function 

```
iter_basic_bands_nb(  
    high,  
    low,  
    atr,  
    multiplier  
)
```



Upper and lower bands at one iteration.

iter_med_price_nb function

```
iter_med_price_nb(  
    high,  
    low  
)
```



Median price at one iteration.

iter_tr_nb function

```
iter_tr_nb(  
    high,  
    low,  
    prev_close  
)
```



True Range (TR) at one iteration.

ma_1d_nb function

```
ma_1d_nb(  
    close,  
    window=14,  
    wtype=0,  
    minp=None,  
    adjust=False  
)
```



Moving average.

For `wtype`, see [WType](#).

ma_nb function

```
ma_nb(  
    close,  
    window=14,  
    wtype=0,  
    minp=None,  
    adjust=False  
)
```

2-dim version of [ma_1d_nb](#).

macd_1d_nb function

```
macd_1d_nb(  
    close,  
    fast_window=12,  
    slow_window=26,  
    signal_window=9,  
    wtype=2,  
    macd_wtype=None,  
    signal_wtype=None,  
    minp=None,  
    macd_minp=None,  
    signal_minp=None,  
    adjust=False,  
    macd_adjust=None,  
    signal_adjust=None  
)
```

MACD.

Returns the MACD and the signal.

For `wtype`, see [WType](#).

macd_hist_1d_nb function

```
macd_hist_1d_nb(  
    macd,  
    signal  
)
```

MACD histogram.

macd_hist_nb function 

```
macd_hist_nb(  
    macd,  
    signal  
)
```

2-dim version of [macd_hist_1d_nb](#).

macd_nb function 

```
macd_nb(  
    close,  
    fast_window=12,  
    slow_window=26,  
    signal_window=9,  
    wtype=2,  
    macd_wtype=None,  
    signal_wtype=None,  
    minp=None,  
    macd_minp=None,  
    signal_minp=None,  
    adjust=False,  
    macd_adjust=None,  
    signal_adjust=None  
)
```

2-dim version of [macd_1d_nb](#).

modes_1d_nb function

```
modes_1d_nb(  
    pivots  
)
```



Modes.

Warning

To be used in plotting. Do not use it as an indicator!

modes_nb function

```
modes_nb(  
    pivots  
)
```



2-dim version of [modes_1d_nb](#).

msd_1d_nb function

```
msd_1d_nb(  
    close,  
    window=14,  
    wtype=0,  
    minp=None,  
    adjust=False,  
    ddof=0  
)
```



Moving standard deviation.

For `wtype`, see [WType](#).

msd_nb function

```
msd_nb(  
    close,  
    window=14,  
    wtype=0,  
    minp=None,  
    adjust=False,  
    ddof=0  
)
```

2-dim version of [msd_1d_nb](#).

obv_1d_nb function

```
obv_1d_nb(  
    close,  
    volume  
)
```

On-Balance Volume (OBV).

obv_nb function

```
obv_nb(  
    close,  
    volume  
)
```

2-dim version of [obv_1d_nb](#).

ols_1d_nb function

```
ols_1d_nb(  
    x,  
    y,  
    window=14,  
    norm_window=None,  
    minp=None,  
    ddof=0,  
    with_zscore=True  
)
```

Rolling Ordinary Least Squares (OLS).

ols_angle_1d_nb function

```
ols_angle_1d_nb(  
    slope  
)
```

OLS angle.

ols_angle_nb function

```
ols_angle_nb(  
    slope  
)
```

2-dim version of [ols_angle_1d_nb](#).

ols_error_1d_nb function

```
ols_error_1d_nb(  
     
```

```
y,  
pred  
)
```

OLS error.

ols_error_nb function 

```
ols_error_nb(  
    y,  
    pred  
)
```

2-dim version of [ols_error_1d_nb](#).

ols_nb function 

```
ols_nb(  
    x,  
    y,  
    window=14,  
    norm_window=None,  
    minp=None,  
    ddof=0,  
    with_zscore=True  
)
```

2-dim version of [ols_1d_nb](#).

ols_pred_1d_nb function 

```
ols_pred_1d_nb(  
    x,  
    slope,
```

```
    intercept  
)
```

OLS prediction.

ols_pred_nb function 

```
ols_pred_nb(  
    x,  
    slope,  
    intercept  
)
```

2-dim version of [ols_pred_1d_nb](#).

pivot_info_1d_nb function 

```
pivot_info_1d_nb(  
    high,  
    low,  
    up_th,  
    down_th  
)
```

Pivot information.

pivot_info_nb function 

```
pivot_info_nb(  
    high,  
    low,  
    up_th,  
    down_th  
)
```

2-dim version of **pivot_info_1d_nb**.

pivot_value_1d_nb function 

```
pivot_value_1d_nb(  
    high,  
    low,  
    last_pivot,  
    last_idx  
)
```



Pivot value.

pivot_value_nb function 

```
pivot_value_nb(  
    high,  
    low,  
    last_pivot,  
    last_idx  
)
```



2-dim version of **pivot_value_1d_nb**.

pivots_1d_nb function 

```
pivots_1d_nb(  
    conf_pivot,  
    conf_idx,  
    last_pivot  
)
```



Pivots.

 Warning

To be used in plotting. Do not use it as an indicator!

pivots_nb function

```
pivots_nb(  
    conf_pivot,  
    conf_idx,  
    last_pivot  
)
```

2-dim version of [pivots_1d_nb](#).

rolling_hurst_1d_nb function

```
rolling_hurst_1d_nb(  
    close,  
    window,  
    method=0,  
    max_lag=20,  
    min_log=1,  
    max_log=2,  
    log_step=0.25,  
    min_chunk=8,  
    max_chunk=100,  
    num_chunks=5,  
    minp=None,  
    stabilize=False  
)
```

Rolling version of [get_hurst_nb](#).

For `method`, see [HurstMethod](#).

rolling_hurst_nb

```
rolling_hurst_nb(  
    close,  
    window,  
    method=0,  
    max_lag=20,  
    min_log=1,  
    max_log=2,  
    log_step=0.25,  
    min_chunk=8,  
    max_chunk=100,  
    num_chunks=5,  
    minp=None,  
    stabilize=False  
)
```

2-dim version of [rolling_hurst_1d_nb](#).

rsi_1d_nb

```
rsi_1d_nb(  
    close,  
    window=14,  
    wtype=3,  
    minp=None,  
    adjust=False  
)
```

RSI.

For `wtype`, see [WType](#).

rsi_nb

```
rsi_nb(  
    close,
```

```
window=14,  
wtype=0,  
minp=None,  
adjust=False  
)
```

2-dim version of [rsi_1d_nb](#).

signal_detection_1d_nb function

```
signal_detection_1d_nb(  
    close,  
    lag=14,  
    factor=1.0,  
    influence=1.0,  
    up_factor=None,  
    down_factor=None,  
    mean_influence=None,  
    std_influence=None  
)
```

Signal detection.

signal_detection_nb function

```
signal_detection_nb(  
    close,  
    lag=14,  
    factor=1.0,  
    influence=1.0,  
    up_factor=None,  
    down_factor=None,  
    mean_influence=None,  
    std_influence=None  
)
```

2-dim version of [signal_detection_1d_nb](#).

stoch_1d_nb function

```
stoch_1d_nb(  
    high,  
    low,  
    close,  
    fast_k_window=14,  
    slow_k_window=3,  
    slow_d_window=3,  
    wtype=0,  
    slow_k_wtype=None,  
    slow_d_wtype=None,  
    minp=None,  
    fast_k_minp=None,  
    slow_k_minp=None,  
    slow_d_minp=None,  
    adjust=False,  
    slow_k_adjust=None,  
    slow_d_adjust=None  
)
```

Stochastic Oscillator.

Returns the fast %K, the slow %K, and the slow %D.

For `wtype`, see [WType](#).

stoch_k_1d_nb function

```
stoch_k_1d_nb(  
    high,  
    low,  
    close,  
    window=14,  
    minp=None  
)
```

Stochastic Oscillator %K.

stoch_k_nb function

```
stoch_k_nb(  
    high,  
    low,  
    close,  
    window=14,  
    minp=None  
)
```

2-dim version of [stoch_k_1d_nb](#).

stoch_nb function

```
stoch_nb(  
    high,  
    low,  
    close,  
    fast_k_window=14,  
    slow_k_window=3,  
    slow_d_window=3,  
    wtype=0,  
    slow_k_wtype=None,  
    slow_d_wtype=None,  
    minp=None,  
    fast_k_minp=None,  
    slow_k_minp=None,  
    slow_d_minp=None,  
    adjust=False,  
    slow_k_adjust=None,  
    slow_d_adjust=None  
)
```

2-dim version of [stoch_1d_nb](#).

supertrend_1d_nb function

```
supertrend_1d_nb(  
    high,  
    low,  
    close,  
    period=7,  
    multiplier=3.0  
)
```

Supertrend.

supertrend_acc_nb function

```
supertrend_acc_nb(  
    in_state  
)
```

Accumulator of [supertrend_nb](#).

Takes a state of type [SuperTrendAIS](#) and returns a state of type [SuperTrendAOS](#).

supertrend_nb function

```
supertrend_nb(  
    high,  
    low,  
    close,  
    period=7,  
    multiplier=3.0  
)
```

2-dim version of [supertrend_1d_nb](#).

tr_1d_nb function

```
tr_1d_nb(  
    high,  
    low,  
    close  
)
```

True Range (TR).

tr_nb function

```
tr_nb(  
    high,  
    low,  
    close  
)
```

2-dim version of [tr_1d_nb](#).

typical_price_1d_nb function

```
typical_price_1d_nb(  
    high,  
    low,  
    close  
)
```

Typical price.

typical_price_nb function

```
typical_price_nb(  
    high,  
    low,  
    close  
)
```

```
    high,  
    low,  
    close  
)
```

2-dim version of **typical_price_1d_nb**.

vwap_1d_nb function 

```
vwap_1d_nb(  
    high,  
    low,  
    close,  
    volume,  
    group_lens  
)
```

Volume-Weighted Average Price (VWAP).

vwap_nb function 

```
vwap_nb(  
    high,  
    low,  
    close,  
    volume,  
    group_lens  
)
```

2-dim version of **vwap_1d_nb**.